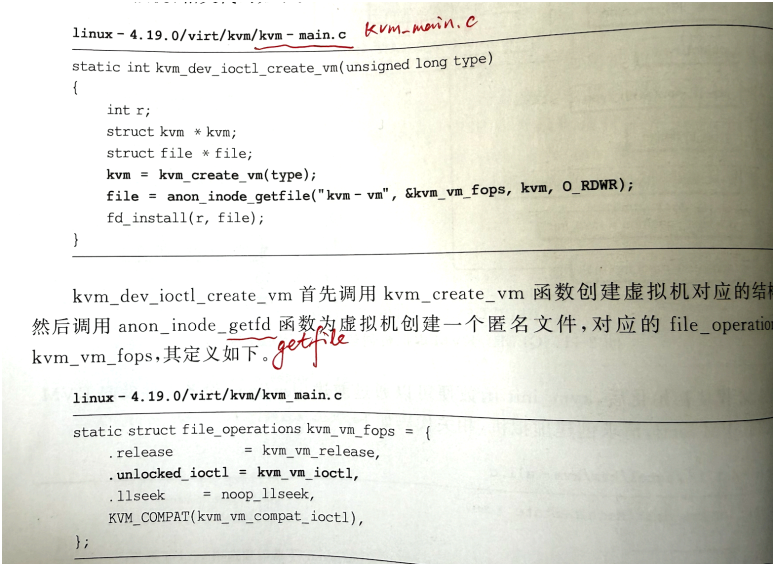
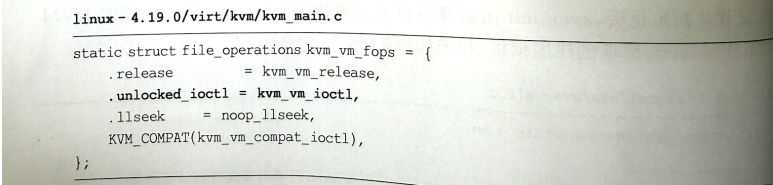
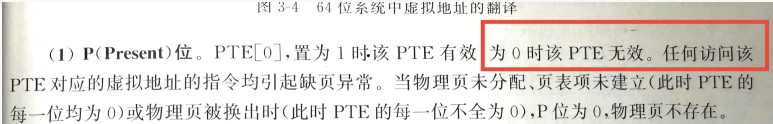
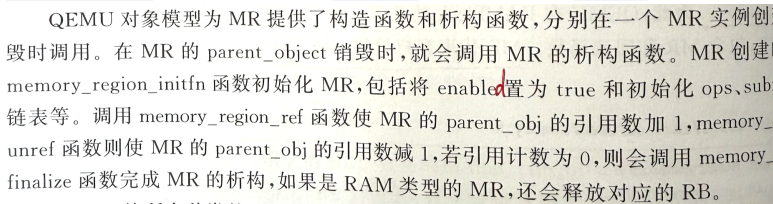
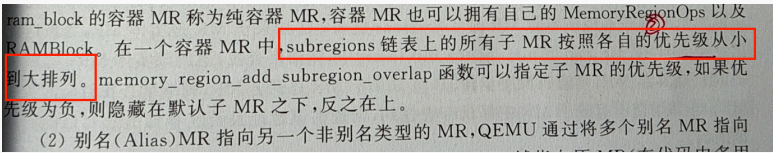


勘误表（印次：2022年8月第3次印刷）

序号	页码	描述
1	P48	文件路径 <code>kvm-main.c</code> 应改为 <code>kvm_main.c</code>；创建匿名文件的函数 <code>anon_inode_getfd</code> 应改为 <code>anon_inode_getfile</code>。  <code>kvm_dev_ioctl_create_vm</code> 首先调用 <code>kvm_create_vm</code> 函数创建虚拟机对应的结构体，然后调用 <code>anon_inode_getfd</code> 函数为虚拟机创建一个匿名文件，对应的 <code>file_operations</code> 为 <code>kvm_vm_fops</code>，其定义如下。 
2	P105	P位为1时，访问其对应虚拟地址的指令未必引起缺页异常，建议改为，。 图 3-4 64 位系统中虚拟地址的翻译 (1) P(Present)位。PTE[0]，置为 1 时该 PTE 有效，为 0 时该 PTE 无效。任何访问该 PTE 对应的虚拟地址的指令均引起缺页异常。当物理页未分配、页表项未建立(此时 PTE 的每一位均为 0)或物理页被换出时(此时 PTE 的每一位不全为 0)，P 位为 0，物理页不存在。 
3	P118	<code>MemoryRegion</code> 结构体的成员为 <code>enabled</code>，原书误写为 <code>enable</code>。 QEMU 对象模型为 MR 提供了构造函数和析构函数，分别在一个 MR 实例创建时调用。在 MR 的 <code>parent_object</code> 销毁时，就会调用 MR 的析构函数。MR 创建时调用 <code>memory_region_initfn</code> 函数初始化 MR，包括将 <code>enabled</code> 置为 <code>true</code> 和初始化 <code>ops</code>、<code>subregions</code> 链表等。调用 <code>memory_region_ref</code> 函数使 MR 的 <code>parent_obj</code> 的引用数加 1，<code>memory_region_unref</code> 函数则使 MR 的 <code>parent_obj</code> 的引用数减 1，若引用计数为 0，则会调用 <code>memory_region_finalize</code> 函数完成 MR 的析构，如果是 RAM 类型的 MR，还会释放对应的 RB。 
4	P119	<code>subregions</code> 的排列顺序为优先级从大到小（即 <code>MemoryRegion.priority</code> 越大，优先级越高，越靠近链表头），原书误写为“优先级从小到大。可参考源代码的插入逻辑(Line 2401-2407)： https://github.com/qemu/qemu/blob/stable-4.1/memory.c。 <code>ram_block</code> 的容器 MR 称为纯容器 MR，容器 MR 也可以拥有自己的 <code>MemoryRegionOps</code> 以及 <code>RAMBlock</code>。在一个容器 MR 中，<code>subregions</code> 链表上的所有子 MR 按照各自的优先级从小到大排列。<code>memory_region_add_subregion_overlap</code> 函数可以指定子 MR 的优先级，如果优先级为负，则隐藏在默认子 MR 之下，反之在上。 (2) 别名(Alias)MR 指向另一个非别名类型的 MR，QEMU 通过将多个别名 MR 指向同一个非别名 MR 来实现别名。 
5	P122	footnote 中的文档链接 "https://qemu.readthedocs.io/en/latest/devel/memory.html" 已失效，建议更新为 "https://qemu.readthedocs.io/en/master/devel/memory.html"。

